

HuggingFace를 활용한

인공지능 웹 애플리케이션 개발

제 5장

오디오 인공지능 웹 프로젝트

이 장의 학습 목표

Whisper 모델로 음성 파일을 텍스트로 변환하는 ASR(자동 음성 인식) API를 구현합니다.

Bark/TTS 모델로 텍스트를 음성으로 합성하는 API와 다양한 출력 형식을 제공합니다.

ASR + 챗봇 + TTS를 파이프라인으로 연결한 완전한 음성 챗봇 서비스를 구축합니다.

UploadFile, BackgroundTasks, StreamingResponse 등 FastAPI 고급 기능을 마스터합니다.

목 차

제5장 오디오 인공지능 웹 프로젝트

★ 이 장의 개요

이 장에서는 오디오 데이터를 다루는 생성형 AI 기술을 FastAPI와 결합하여 음성 인식(ASR), 음성 합성(TTS), 음성 챗봇 등의 기능을 직접 구현합니다. Whisper, Bark, TTS 모델들을 사용하여 음성 → 텍스트, 텍스트 → 음성의 흐름을 이해하고, 업로드된 오디오 파일을 실시간으로 처리하는 RESTful API도 함께 구축합니다. 오디오 기반 AI 기술이 웹 서비스에 어떻게 접목될 수 있는지, 사용자 음성을 입력받아 챗봇처럼 응답하거나 자막을 자동 생성하는 방식 등을 단계별로 실습합니다.

음성(오디오)은 텍스트, 이미지와 함께 AI가 처리하는 가장 핵심적인 데이터 유형 중 하나입니다. 스마트 스피커, 음성 비서, 자동 자막, 오디오북 등 실생활 속 AI 음성 기술은 이미 수십억 명의 일상에 자리 잡았습니다. 이러한 기술의 근간에는 ASR(Automatic Speech Recognition, 자동 음성 인식)과 TTS(Text-To-Speech, 음성 합성)가 있습니다.

Hugging Face는 OpenAI의 Whisper, Suno의 Bark, Microsoft의 SpeechT5 등 최고 수준의 오디오 AI 모델을 무료로 제공합니다. 이 모델들을 FastAPI와 결합하면, 기업 수준의 음성 처리 서비스를 빠르게 구축할 수 있습니다.

📁 5장 프로젝트 디렉터리 구조

chapter5-audio-ai/

```

├── main.py                # FastAPI 앱 진입점
├── requirements.txt
├── .env                  # 환경변수 설정
├── routers/
│   ├── asr.py            # 5.1 음성 인식 라우터
│   ├── tts.py            # 5.2 음성 합성 라우터
│   ├── voice_chat.py     # 5.3 음성 챗봇 라우터
│   └── audio_upload.py   # 5.4 오디오 업로드 라우터

```

```

├── services/
│   ├── asr_service.py      # Whisper ASR 서비스
│   ├── tts_service.py      # Bark/SpeechT5 TTS 서비스
│   ├── chatbot_service.py  # 텍스트 챗봇 서비스
│   └── audio_utils.py      # 오디오 파일 처리 유틸
├── models/
│   └── schemas.py          # Pydantic 스키마
├── temp/                  # 임시 파일 저장소
└── output/                # 생성된 오디오 파일

```

5장 전체에서 사용하는 requirements.txt를 먼저 살펴봅니다.

```

# requirements.txt

# Web Framework
fastapi==0.111.0
uvicorn[standard]==0.29.0
python-multipart==0.0.9 # UploadFile 처리에 필수
aiofiles==23.2.1        # 비동기 파일 I/O

# AI / Audio Core
torch==2.3.0
transformers==4.41.0
openai-whisper==20231117 # Whisper (openai 패키지)
# 또는 Hugging Face Whisper:
# transformers[audio]    # Whisper via HF pipeline

# TTS 모델
bark==0.1.5             # Suno Bark TTS
TTS==0.22.0              # Coqui TTS (SpeechT5 래핑)
soundfile==0.12.1       # 오디오 파일 읽기/쓰기
librosa==0.10.2          # 오디오 전처리/분석

# 오디오 포맷 변환
pydub==0.25.1           # MP3/WAV 변환 (ffmpeg 필요)

```

```
scipy==1.13.0          # 과학 연산 (오디오 신호 처리)
```

```
# 유틸리티
```

```
pydantic==2.7.1
```

```
python-dotenv==1.0.1
```

```
numpy==1.26.4
```

```
httpx==0.27.0
```

⚠ ffmpeg 설치 필수

오디오 포맷 변환(MP3 → WAV 등)과 pydub 라이브러리 사용을 위해 ffmpeg가 필요합니다.

Ubuntu/Debian:

```
$ sudo apt update && sudo apt install ffmpeg
```

macOS (Homebrew):

```
$ brew install ffmpeg
```

Windows:

```
$ winget install ffmpeg
```

또는 <https://ffmpeg.org/download.html> 에서 직접 다운로드
(시스템 PATH에 추가 필요)

설치 확인:

```
$ ffmpeg -version
```

5.1 Whisper로 음성 인식 API

OpenAI가 2022년 공개한 Whisper는 현재 세계 최고 수준의 다국어 음성 인식 모델입니다. 영어, 한국어를 포함한 99개 언어를 지원하며, Hugging Face Hub에도 공개되어 있어 Pipeline API 또는 openai-whisper 패키지로 바로 사용할 수 있습니다.

5.1.1 Hugging Face openai/whisper 모델 소개

Whisper는 680,000시간의 다국어 오디오 데이터로 학습된 Transformer 기반의 Encoder-Decoder 모델입니다. 음성 신호를 먼저 80채널 Log-Mel Spectrogram으로 변환한 후 인코더에 입력하고, 디코더가 자동 회귀 방식으로 텍스트를 생성합니다.

모델 크기	파라미터	디스크 크기	특징
tiny	39M	~ 75MB	가장 빠름. 실험/체험용.
base	74M	~ 145MB	속도와 정확도 균형. 학습/테스트 추천.
small	244M	~ 480MB	일반 사용에 충분한 품질.
medium	769M	~ 1.5GB	고품질. GPU 권장.
large-v3	1550M	~ 3.1GB	최고 정확도. A100 GPU 권장.

💡 팁: 작은 버전으로도 충분히 체험 가능

CPU 환경에서는 tiny 또는 base 모델을 권장합니다.

tiny: 약 10초 길이 오디오를 CPU에서 15~30초 내 처리 가능

base: CPU에서 30~60초 처리 시간. 한국어 인식도 양호.

GPU(T4) 환경에서는 medium 이상을 사용하면 실시간(1초 오디오 < 1초 처리) 가능.

Hugging Face Inference API를 사용하면 로컬 GPU 없이도 large-v3 이용 가능:

```
from huggingface_hub import InferenceClient
client = InferenceClient(token='hf_...')
result = client.automatic_speech_recognition('audio.wav')
```

5.1.2 Whisper의 장점: 언어 감지 + 정확한 자막 생성

Whisper가 다른 ASR 모델 대비 특별히 강점을 갖는 영역은 다음과 같습니다.

Whisper의 장점	구체적인 내용
자동 언어 감지	입력 오디오 첫 30초를 분석하여 99개 언어 중 자동 감지. 별도 언어 설정 불필요.
타임스탬프 지원	단어/문장 단위 타임스탬프 생성 가능. 자막(SRT/VTX) 자동 생성이 용.
잡음 환경 강인성	배경 음악, 잡음이 있는 환경에서도 높은 인식률. 6dB 이상 SNR에서 90%+ 정확도.
다국어 번역	음성을 인식하면서 동시에 영어로 번역 가능 (translate 태스크).
긴 오디오 처리	30초 단위로 청크(Chunk) 분할 후 연속 처리. 수 시간 분량 오디오도 처리 가능.
구어체 강인성	말끝 흐림, 겹쳐 말하기, 방언 등 자연스러운 대화 인식에 강함.

Whisper가 내부적으로 오디오를 처리하는 과정을 단계별로 살펴봅니다.

1. 오디오 전처리: 입력 오디오를 16kHz 모노(Mono) WAV로 변환
2. Log-Mel Spectrogram 변환: 80채널, 30초 단위 스펙트로그램 생성 (3,000 프레임 × 80채널)
3. 인코더(Encoder): Transformer 인코더로 오디오 특징(Feature) 추출
4. 언어 감지: 디코더 첫 토큰 예측으로 언어 자동 판별
5. 디코더(Decoder): 자기 회귀 방식으로 텍스트 토큰 순서대로 예측
6. 타임스탬프 정렬: 각 단어의 시작/종료 시간 계산 (DTW 알고리즘)

5.1.3 오디오 파일(mp3, wav 등) → 텍스트 변환 FastAPI API 만들기

Whisper ASR 서비스 로직을 먼저 구현하겠습니다. 서비스 레이어에서 Hugging Face Pipeline 방식과 openai-whisper 패키지 방식을 모두 지원하도록 설계합니다.

`services/asr_service.py`

```
# services/asr_service.py
```

```
import os
import time
import torch
import tempfile
import soundfile as sf
import numpy as np
from pathlib import Path
from typing import Optional, Dict, Any

# Hugging Face Pipeline 방식 (권장)
from transformers import pipeline, AutoModelForSpeechSeq2Seq, AutoProcessor

class ASRService:
    """Whisper 기반 자동 음성 인식(ASR) 서비스"""

    SUPPORTED_MODELS = {
        'tiny': 'openai/whisper-tiny',
        'base': 'openai/whisper-base',
        'small': 'openai/whisper-small',
        'medium': 'openai/whisper-medium',
        'large': 'openai/whisper-large-v3',
    }

    def __init__(self, model_size: str = 'base'):
        self.model_size = model_size
        self.pipe = None
        self.device = 'cuda:0' if torch.cuda.is_available() else 'cpu'
        self.torch_dtype = (
            torch.float16 if torch.cuda.is_available() else torch.float32
        )

    def load_model(self):
        """Whisper 파이프라인 로드"""
        model_id = self.SUPPORTED_MODELS.get(
            self.model_size, 'openai/whisper-base'
        )
```



```
print(f'[ASR] Loading Whisper model: {model_id}')
print(f'[ASR] Device: {self.device}, dtype: {self.torch_dtype}')

model = AutoModelForSpeechSeq2Seq.from_pretrained(
    model_id,
    torch_dtype=self.torch_dtype,
    low_cpu_mem_usage=True,
    use_safetensors=True,
)
model.to(self.device)

processor = AutoProcessor.from_pretrained(model_id)

self.pipe = pipeline(
    task='automatic-speech-recognition',
    model=model,
    tokenizer=processor.tokenizer,
    feature_extractor=processor.feature_extractor,
    torch_dtype=self.torch_dtype,
    device=self.device,
)

print(f'[ASR] Whisper {self.model_size} loaded successfully.')

def transcribe(
    self,
    audio_path: str,
    language: Optional[str] = None,
    task: str = 'transcribe',
    return_timestamps: bool = False,
    chunk_length_s: int = 30,
) -> Dict[str, Any]:
    """
    오디오 파일을 텍스트로 변환합니다.

    Args:
        audio_path: 오디오 파일 경로 (wav, mp3, m4a 등)
        language: 언어 코드 (None이면 자동 감지, 'ko'=한국어, 'en'=영어)
```

```

task: 'transcribe'(인식) 또는 'translate'(영어로 번역)
return_timestamps: True면 단어 단위 타임스탬프 포함
chunk_length_s: 청크 분할 길이 (초). 긴 오디오 처리 시 사용.
"""

if self.pipe is None:
    raise RuntimeError('ASR 모델이 로드되지 않았습니다.')

start_time = time.time()

# generate_kwargs 구성
generate_kwargs = { 'task': task }
if language:
    generate_kwargs['language'] = language

# 긴 오디오를 위한 청크 파이프라인 설정
result = self.pipe(
    audio_path,
    generate_kwargs=generate_kwargs,
    return_timestamps=return_timestamps,
    chunk_length_s=chunk_length_s,
    stride_length_s=chunk_length_s // 6, # 청크 간 오버랩
)

elapsed = round(time.time() - start_time, 2)

# 오디오 파일 길이 측정
try:
    audio_data, sample_rate = sf.read(audio_path)
    audio_duration = round(len(audio_data) / sample_rate, 2)
except Exception:
    audio_duration = None

return {
    'text': result['text'].strip(),
    'chunks': result.get('chunks', []),
    'language': language or 'auto-detected',
    'task': task,

```

```

        'audio_duration_sec': audio_duration,
        'processing_time_sec': elapsed,
        'model': f'openai/whisper-{self.model_size}',
        'realtime_factor': (
            round(elapsed / audio_duration, 2)
            if audio_duration else None
        ),
    }

def transcribe_bytes(
    self,
    audio_bytes: bytes,
    suffix: str = '.wav',
    **kwargs
) -> Dict[str, Any]:
    """바이트 데이터로 직접 변환 (임시 파일 사용)"""
    with tempfile.NamedTemporaryFile(suffix=suffix, delete=False) as f:
        f.write(audio_bytes)
        tmp_path = f.name
    try:
        return self.transcribe(tmp_path, **kwargs)
    finally:
        os.unlink(tmp_path) # 임시 파일 삭제

def generate_srt(
    self,
    audio_path: str,
    language: Optional[str] = None,
) -> str:
    """SRT 형식 자막 파일 생성"""
    result = self.transcribe(
        audio_path,
        language=language,
        return_timestamps=True,
    )
    chunks = result.get('chunks', [])
    if not chunks:
        return f'1\n00:00:00,000 --> 00:10:00,000\n{result["text"]}\n'

```

```

srt_lines = []
for i, chunk in enumerate(chunks, 1):
    ts = chunk.get('timestamp', (0, 1))
    start = self._sec_to_srt_time(ts[0] or 0)
    end = self._sec_to_srt_time(ts[1] or (ts[0] or 0) + 2)
    srt_lines.append(f'{i}')
    srt_lines.append(f'{start} --> {end}')
    srt_lines.append(chunk['text'].strip())
    srt_lines.append("")
return '\n'.join(srt_lines)

@staticmethod
def _sec_to_srt_time(seconds: float) -> str:
    """초 → SRT 타임코드 변환 (hh:mm:ss,mmm)"""
    ms = int((seconds % 1) * 1000)
    s = int(seconds) % 60
    m = (int(seconds) // 60) % 60
    h = int(seconds) // 3600
    return f'{h:02d}:{m:02d}:{s:02d},{ms:03d}'

# 싱글톤 인스턴스
asr_service = ASRService(model_size='base')

```

5.1.4 torchaudio, ffmpeg 설치 및 파일 처리 방법 설명

오디오 파일 처리에는 여러 도구가 관여합니다. 각 도구의 역할을 명확히 이해해야 안정적인 서비스를 구축할 수 있습니다.

라이브러리 / 도구	역할 및 사용 상황
ffmpeg	오디오/비디오 포맷 변환의 핵심 엔진. MP3→WAV, M4A→WAV 등. pydub의 백엔드.
pydub	Python ffmpeg 래퍼. 오디오 슬라이싱, 볼륨 조절, 포맷 변환을 파이썬 코드로 처리.
soundfile	WAV, FLAC, OGG 파일 읽기/쓰기. numpy 배열로 바로 변환. 빠르고 가벼움.

라이브러리 / 도구	역할 및 사용 상황
librosa	오디오 분석 및 특징 추출. 피치 감지, 스펙트로그램 생성 등. 과학 논문 활용.
torchaudio	PyTorch 기반 오디오 I/O. Whisper HF 파이프라인 내부에서 전처리 활용.
scipy.io.wavfile	간단한 WAV 파일 읽기/쓰기. 추가 의존성 없이 사용 가능.

services/audio_utils.py - 오디오 전처리 유틸리티

```
# services/audio_utils.py
import os
import uuid
import numpy as np
from pathlib import Path
from typing import Tuple, Optional

try:
    from pydub import AudioSegment
    PYDUB_AVAILABLE = True
except ImportError:
    PYDUB_AVAILABLE = False

import soundfile as sf

TEMP_DIR = Path('temp')
OUTPUT_DIR = Path('output')
TEMP_DIR.mkdir(exist_ok=True)
OUTPUT_DIR.mkdir(exist_ok=True)

SUPPORTED_AUDIO_FORMATS = {'.wav', '.mp3', '.m4a', '.ogg', '.flac', '.webm', '.mp4'}
MAX_FILE_SIZE_MB = 50

def validate_audio_file(filename: str, size_bytes: int) -> None:
    """오디오 파일 유효성 검사"""
    ext = Path(filename).suffix.lower()
    if ext not in SUPPORTED_AUDIO_FORMATS:
```

```

    raise ValueError(
        f'지원하지 않는 파일 형식: {ext}. '
        f'지원 형식: {SUPPORTED_AUDIO_FORMATS}'
    )
if size_bytes > MAX_FILE_SIZE_MB * 1024 * 1024:
    raise ValueError(
        f'파일 크기 초과: {size_bytes / 1024 / 1024:.1f}MB. '
        f'최대 {MAX_FILE_SIZE_MB}MB까지 허용.'
    )

def convert_to_wav(
    input_path: str,
    output_path: Optional[str] = None,
    sample_rate: int = 16000,
    channels: int = 1,
) -> str:
    """
    오디오 파일을 Whisper 입력 형식(16kHz, 모노 WAV)으로 변환합니다.
    """
    if output_path is None:
        output_path = str(TEMP_DIR / f'{uuid.uuid4()}.wav')

    if PYDUB_AVAILABLE:
        audio = AudioSegment.from_file(input_path)
        audio = audio.set_frame_rate(sample_rate)
        audio = audio.set_channels(channels)
        audio.export(output_path, format='wav')
    else:
        # soundfile로 읽고 scipy로 리샘플링
        import scipy.signal as signal
        data, orig_sr = sf.read(input_path)
        if orig_sr != sample_rate:
            num_samples = int(len(data) * sample_rate / orig_sr)
            data = signal.resample(data, num_samples)
        if data.ndim > 1:
            data = data.mean(axis=1) # 스테레오 → 모노
        sf.write(output_path, data, sample_rate)
    return output_path

```

```

def get_audio_info(audio_path: str) -> dict:
    """오디오 파일 기본 정보 반환"""
    try:
        data, sr = sf.read(audio_path)
        duration = len(data) / sr
        channels = 1 if data.ndim == 1 else data.shape[1]
        return {
            'duration_sec': round(duration, 2),
            'sample_rate': sr,
            'channels': channels,
            'samples': len(data),
            'format': Path(audio_path).suffix.lower(),
        }
    except Exception as e:
        return {'error': str(e)}

def save_upload_file(content: bytes, filename: str) -> str:
    """업로드 파일을 임시 디렉터리에 저장"""
    ext = Path(filename).suffix.lower()
    unique_name = f'{uuid.uuid4()}{ext}'
    save_path = str(TEMP_DIR / unique_name)
    with open(save_path, 'wb') as f:
        f.write(content)
    return save_path

def cleanup_temp_files(max_age_sec: int = 3600):
    """1시간 이상 된 임시 파일 정리"""
    import time
    now = time.time()
    for f in TEMP_DIR.iterdir():
        if f.is_file() and now - f.stat().st_mtime > max_age_sec:
            f.unlink(missing_ok=True)

```

5.1.5 비동기 업로드 파일 처리 (UploadFile 활용)

FastAPI의 UploadFile은 업로드된 파일을 비동기로 처리하는 핵심 클래스입니다. 파일 메타

데이터(이름, 크기, M I M E 타입)에 접근하고, `async/await`로 파일 내용을 읽을 수 있습니다.

UploadFile 속성/메서드	설명
<code>filename</code>	업로드된 파일의 원본 파일명 (문자열)
<code>content_type</code>	M I M E 타입 (예: 'audio/wav', 'audio/mpeg')
<code>size</code>	파일 크기(바이트). None일 수 있으므로 직접 읽어서 측정 권장.
<code>await file.read()</code>	파일 전체 내용을 bytes로 비동기 읽기
<code>await file.read(size)</code>	지정 바이트 수만큼 읽기 (스트리밍 처리용)
<code>await file.seek(0)</code>	파일 포인터를 처음으로 이동 (재읽기 시 필요)
<code>await file.close()</code>	파일 스트림 종료. FastAPI가 자동 처리하지만 명시 권장.
<code>file.file</code>	실제 <code>SpooledTemporaryFile</code> 객체. 저수준 파일 작업 시 활용.

routers/asr.py - 음성 인식 라우터

```
# routers/asr.py
import os
from pathlib import Path
from fastapi import (
    APIRouter, UploadFile, File, Form,
    HTTPException, status, BackgroundTasks,
)
from fastapi.responses import PlainTextResponse
from typing import Optional
from models.schemas import ASRResponse, SRTResponse
from services.asr_service import asr_service
from services.audio_utils import (
    validate_audio_file, save_upload_file,
    convert_to_wav, cleanup_temp_files, get_audio_info
)

router = APIRouter(prefix='/asr', tags=['Speech Recognition (ASR)'])

@router.post(
```



```

'/transcribe',
response_model=ASRResponse,
summary='오디오 파일을 텍스트로 변환 (STT)',
description=""
WAV, MP3, M4A, OGG 등 오디오 파일을 업로드하면
Whisper 모델이 텍스트로 변환합니다.
language를 비워두면 자동으로 언어를 감지합니다.
""
)
async def transcribe_audio(
    background_tasks: BackgroundTasks,
    file: UploadFile = File(
        ...,
        description='오디오 파일 (wav/mp3/m4a/ogg/flac, 최대 50MB)',
    ),
    language: Optional[str] = Form(
        None,
        description='언어 코드 (ko, en, ja 등). 비우면 자동 감지.',
    ),
    task: str = Form(
        'transcribe',
        description='transcribe(인식) 또는 translate(영어로 번역)',
    ),
    return_timestamps: bool = Form(
        False,
        description='True면 단어 단위 타임스탬프 포함',
    ),
):
    # 1. 파일 유효성 검사
    content = await file.read()
    try:
        validate_audio_file(file.filename, len(content))
    except ValueError as e:
        raise HTTPException(
            status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
            detail=str(e)
        )

```

2. 임시 파일 저장

```
tmp_path = save_upload_file(content, file.filename)
```

3. WAV 변환 (필요 시)

```
ext = Path(file.filename).suffix.lower()
```

```
if ext != '.wav':
```

```
    wav_path = tmp_path.replace(ext, '.wav')
```

```
    convert_to_wav(tmp_path, wav_path)
```

```
    process_path = wav_path
```

```
else:
```

```
    process_path = tmp_path
```

4. 오디오 정보 수집

```
audio_info = get_audio_info(process_path)
```

5. ASR 추론

```
try:
```

```
    result = asr_service.transcribe(
```

```
        audio_path=process_path,
```

```
        language=language,
```

```
        task=task,
```

```
        return_timestamps=return_timestamps,
```

```
    )
```

```
except Exception as e:
```

```
    raise HTTPException(
```

```
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
```

```
        detail=f'음성 인식 처리 중 오류: {str(e)}'
```

```
    )
```

```
finally:
```

6. 임시 파일 백그라운드 정리

```
background_tasks.add_task(os.unlink, tmp_path)
```

```
if ext != '.wav' and os.path.exists(process_path):
```

```
    background_tasks.add_task(os.unlink, process_path)
```

```
return ASRResponse(
```

```
    filename=file.filename,
```

```
    transcribed_text=result['text'],
```

```

        language=result['language'],
        task=task,
        audio_duration_sec=audio_info.get('duration_sec'),
        processing_time_sec=result['processing_time_sec'],
        realtime_factor=result.get('realtime_factor'),
        model_used=result['model'],
        chunks=result.get('chunks', []) if return_timestamps else [],
    )

@router.post(
    '/generate-srt',
    response_class=PlainTextResponse,
    summary='SRT 자막 파일 생성',
    description='오디오 파일로부터 SRT 형식 자막을 생성합니다.',
)
async def generate_srt(
    background_tasks: BackgroundTasks,
    file: UploadFile = File(...),
    language: Optional[str] = Form(None),
):
    content = await file.read()
    try:
        validate_audio_file(file.filename, len(content))
    except ValueError as e:
        raise HTTPException(status_code=422, detail=str(e))

    tmp_path = save_upload_file(content, file.filename)
    ext = Path(file.filename).suffix.lower()

    if ext != '.wav':
        wav_path = tmp_path.replace(ext, '.wav')
        convert_to_wav(tmp_path, wav_path)
        process_path = wav_path
    else:
        process_path = tmp_path

```

```

try:
    srt_content = asr_service.generate_srt(process_path, language)
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))
finally:
    background_tasks.add_task(os.unlink, tmp_path)
    if ext != '.wav':
        background_tasks.add_task(os.unlink, process_path)

return PlainTextResponse(
    content=srt_content,
    media_type='text/plain; charset=utf-8',
    headers={'Content-Disposition':
        f'attachment; filename="{Path(file.filename).stem}.srt"'},
)

@router.get('/health', summary='ASR 모델 상태 확인')
async def asr_health():
    return {
        'model_loaded': asr_service.pipe is not None,
        'model_size': asr_service.model_size,
        'device': asr_service.device,
        'supported_formats': list(SUPPORTED_AUDIO_FORMATS),
    }

```

models/schemas.py - ASR 스키마

```

# models/schemas.py (ASR 관련)
from pydantic import BaseModel, Field
from typing import Optional, List, Any

class TimestampChunk(BaseModel):
    text: str
    timestamp: List[Optional[float]] # [시작초, 종료초]

class ASRResponse(BaseModel):

```

filename: str
transcribed_text: str
language: str
task: str
audio_duration_sec: Optional[float] = None
processing_time_sec: float
realtime_factor: Optional[float] = None # 처리시간/오디오길이
model_used: str
chunks: List[Any] = [] # 타임스탬프 청크 (옵션)

테스트 실행 방법을 살펴봅니다.

curl로 오디오 파일 업로드 및 음성 인식
\$ curl -X POST http://localhost:8000/asr/transcribe ₩
-F 'file=@sample_audio.mp3' ₩
-F 'language=ko' ₩
-F 'task=transcribe' ₩
-F 'return_timestamps=false'
응답 예시:
{
"filename": "sample_audio.mp3",
"transcribed_text": "안녕하세요. 오늘은 인공지능 웹 개발에 대해 설명드리겠습니다.",
"language": "ko",
"task": "transcribe",
"audio_duration_sec": 8.32,
"processing_time_sec": 4.17,
"realtime_factor": 0.50,
"model_used": "openai/whisper-base",
"chunks": []
}
SRT 자막 생성 테스트
\$ curl -X POST http://localhost:8000/asr/generate-srt ₩
-F 'file=@lecture.mp3' ₩
-o lecture_subtitle.srt

5.2 텍스트 → 음성 API (Bark 또는 TTS)

TTS(Text-To-Speech, 음성 합성)는 텍스트를 자연스러운 음성 오디오로 변환하는 기술입니다. 이 절에서는 Suno Bark와 Microsoft SpeechT5, 두 가지 접근 방식으로 TTS API를 구현합니다. 두 모델은 성능, 속도, 메모리 요구사항이 서로 다르므로, 운영 환경에 맞게 선택할 수 있도록 모두 구현합니다.

5.2.1 Hugging Face suno/bark 또는 SpeechT5 모델 활용

Bark는 Suno AI가 2023년 공개한 오픈소스 TTS 모델로, 다양한 음성 캐릭터, 감정 표현, 비언어 사운드(웃음, 울음, 노래 등)를 생성할 수 있는 혁신적인 모델입니다. SpeechT5는 Microsoft의 T5 기반 다목적 음성 처리 모델로, 더 빠른 처리 속도와 안정성을 제공합니다.

특성	suno/bark	microsoft/speecht5_tts
모델 크기	~ 1.7GB (3개 서브모델)	~ 400MB
처리 속도	느림 (CPU: 수 분)	빠름 (CPU: 수 초)
음성 품질	매우 자연스러움	자연스러움
감정 표현	매우 다양 (특수 태그)	제한적
음성 캐릭터	100+ 프리셋	화자 임베딩 필요
비언어 사운드	웃음, 울음, 노래 가능	불가
다국어 지원	영어 최적화 (기타 언어 실험적)	다국어 화자 임베딩 활용
GPU 요구사항	강력히 권장	선택 사항

5.2.2 텍스트 입력 → 음성 합성 → 오디오 스트리밍 응답

`services/tts_service.py`

```
# services/tts_service.py
import os
import io
import time
import uuid
```

```
import torch
import numpy as np
import soundfile as sf
from pathlib import Path
from typing import Optional, Literal

OUTPUT_DIR = Path('output')
OUTPUT_DIR.mkdir(exist_ok=True)

class BarkTTSService:
    """Suno Bark 기반 고품질 음성 합성 서비스"""

    VOICE_PRESETS = {
        # 영어 남성
        'en_male_1': 'v2/en_speaker_0',
        'en_male_2': 'v2/en_speaker_1',
        'en_male_deep': 'v2/en_speaker_3',
        # 영어 여성
        'en_female_1': 'v2/en_speaker_9',
        'en_female_2': 'v2/en_speaker_7',
        # 한국어
        'ko_male_1': 'v2/ko_speaker_0',
        'ko_female_1': 'v2/ko_speaker_2',
        # 기타
        'ja_male': 'v2/ja_speaker_0',
        'zh_male': 'v2/zh_speaker_0',
    }

    # Bark 감정/스타일 태그
    EMOTION_TAGS = {
        'neutral': '',
        'laugh': '[laughter]',
        'sigh': '[sighs]',
        'music': '[music]',
        'applause': '[applause]',
        'hesitate': '[hesitates]',
```

```

}

def __init__(self):
    self.processor = None
    self.model = None
    self.device = 'cuda' if torch.cuda.is_available() else 'cpu'
    self.sample_rate = 24000 # Bark 기본 샘플링 레이트

def load_model(self):
    from transformers import AutoProcessor, BarkModel
    model_id = 'suno/bark-small' # 'suno/bark' (더 큰 모델)
    print(f'[TTS-Bark] Loading model: {model_id}')
    self.processor = AutoProcessor.from_pretrained(model_id)
    self.model = BarkModel.from_pretrained(
        model_id,
        torch_dtype=torch.float16 if self.device=='cuda' else torch.float32,
    ).to(self.device)
    self.model.enable_cpu_offload() # 메모리 절약
    print('[TTS-Bark] Model loaded.')

def synthesize(
    self,
    text: str,
    voice_preset: str = 'en_male_1',
    emotion: str = 'neutral',
    temperature: float = 0.7,
) -> np.ndarray:
    """텍스트를 음성 numpy 배열로 변환"""
    if self.model is None:
        raise RuntimeError('Bark 모델이 로드되지 않았습니다.')

    # 감정 태그 추가
    emotion_tag = self.EMOTION_TAGS.get(emotion, "")
    input_text = f'{emotion_tag} {text}'.strip() if emotion_tag else text

    # 음성 프리셋
    preset = self.VOICE_PRESETS.get(voice_preset, 'v2/en_speaker_0')

```



```
# 입력 처리
inputs = self.processor(
    input_text,
    voice_preset=preset,
)
inputs = {k: v.to(self.device) for k, v in inputs.items()}

# 생성
with torch.no_grad():
    audio_array = self.model.generate(
        **inputs,
        do_sample=True,
        temperature=temperature,
    )

    return audio_array.cpu().numpy().squeeze()

def save_to_wav(self, audio_array: np.ndarray, filepath: str) -> str:
    sf.write(filepath, audio_array, self.sample_rate)
    return filepath

def to_bytes(self, audio_array: np.ndarray, fmt: str = 'wav') -> bytes:
    """numpy 배열을 bytes로 변환"""
    buf = io.BytesIO()
    sf.write(buf, audio_array, self.sample_rate, format=fmt.upper())
    return buf.getvalue()

class SpeechT5Service:
    """Microsoft SpeechT5 기반 빠른 음성 합성 서비스"""

    def __init__(self):
        self.processor = None
        self.model = None
        self.vocoder = None
```

```
self.speaker_embeddings = None

self.device = 'cuda' if torch.cuda.is_available() else 'cpu'

self.sample_rate = 16000

def load_model(self):
    from transformers import (
        SpeechT5Processor, SpeechT5ForTextToSpeech,
        SpeechT5HifiGan
    )
    from datasets import load_dataset

    print('[TTS-SpeechT5] Loading model...')
    self.processor = SpeechT5Processor.from_pretrained(
        'microsoft/speecht5_tts'
    )
    self.model = SpeechT5ForTextToSpeech.from_pretrained(
        'microsoft/speecht5_tts'
    ).to(self.device)
    self.vocoder = SpeechT5HifiGan.from_pretrained(
        'microsoft/speecht5_hifigan'
    ).to(self.device)

    # 기본 화자 임베딩 로드 (CMU ARCTIC 데이터셋)
    embeddings_dataset = load_dataset(
        'Matthijs/cmu-arctic-xvectors', split='validation'
    )
    # 화자 인덱스 7306 = 영어 여성 화자
    self.speaker_embeddings = torch.tensor(
        embeddings_dataset[7306]['xvector']
    ).unsqueeze(0).to(self.device)
    print('[TTS-SpeechT5] Model loaded.')

def synthesize(self, text: str) -> np.ndarray:
    if self.model is None:
        raise RuntimeError('SpeechT5 모델이 로드되지 않았습니다.')

    inputs = self.processor(
```

```

        text=text, return_tensors='pt'
    ).to(self.device)

    with torch.no_grad():
        speech = self.model.generate_speech(
            inputs['input_ids'],
            self.speaker_embeddings,
            vocoder=self.vocoder,
        )
        return speech.cpu().numpy()

def to_bytes(self, audio_array: np.ndarray) -> bytes:
    buf = io.BytesIO()
    sf.write(buf, audio_array, self.sample_rate, format='WAV')
    return buf.getvalue()

# 싱글턴
bark_tts = BarkTTSService()
speecht5_tts = SpeechT5Service()

```

5.2.3 음성 캐릭터 선택, 샘플링 속도, 감정 톤 조정

Bark 모델은 특수 텍스트 태그를 통해 다양한 음성 스타일과 감정을 제어할 수 있습니다. 이는 Bark의 가장 독특한 특징으로, 단순한 TTS를 넘어 오디오 드라마, 감정 표현 콘텐츠 생성 등에 활용됩니다.

Bark 특수 태그	효과 및 사용 예시
[laughter]	웃음 소리 삽입. '그거 정말 웃겨! [laughter] 하하하'
[sighs]	한숨 소리 삽입. '[sighs] 이걸 너무 어렵군요...'
[music]	배경 음악 분위기. '[music] 아름다운 선율이 흐릅니다'
[applause]	박수 소리. '수고하셨습니다! [applause]'
[gasps]	놀람 소리. '[gasps] 세상에, 믿을 수가 없네요!'
[clears throat]	헛기침. '[clears throat] 이제 발표를 시작합니다.'

Bark 특수 태그	효과 및 사용 예시
♪ ... ♪	노래 형식. '♪ 생일 축하합니다 ♪'

음성 합성 속도와 음조를 조정하는 파라미터들도 중요합니다.

파라미터	역할	권장 범위
temperature (text)	텍스트 코덱 토큰 샘플링 다양성. 낮을수록 안정적.	0.6 ~ 0.9
temperature (waveform)	파형 생성 다양성. 낮을수록 깔끔한 발음.	0.6 ~ 0.8
fine_temperature	세밀한 음향 조정. 낮을수록 노이즈 감소.	0.4 ~ 0.6
voice_preset	화자 스타일 선택. 언어별 프리셋 제공.	v2/ko_speaker_0 등

5.2.4 반환 형식: .wav / .mp3 / base64 지원

API 사용자가 다양한 환경에서 오디오를 활용할 수 있도록, WAV, MP3 스트리밍 응답과 Base64 인코딩 JSON 응답을 모두 지원합니다.

routers/tts.py

```
# routers/tts.py
import base64
import io
import uuid
from pathlib import Path
from typing import Optional, Literal

from fastapi import APIRouter, HTTPException, status, BackgroundTasks
from fastapi.responses import StreamingResponse, FileResponse
from pydantic import BaseModel, Field

from services.tts_service import bark_tts, speecht5_tts
import soundfile as sf
import numpy as np
```

```
router = APIRouter(prefix='/tts', tags=['Text-to-Speech (TTS)'])

OUTPUT_DIR = Path('output')
OUTPUT_DIR.mkdir(exist_ok=True)

class TTSRequest(BaseModel):
    text: str = Field(
        ...,
        min_length=1,
        max_length=500,
        description='음성으로 변환할 텍스트 (최대 500자)',
    )
    engine: Literal['bark', 'speecht5'] = Field(
        'speecht5',
        description='TTS 엔진 선택 (bark=고품질/느림, speecht5=빠름)',
    )
    voice_preset: Optional[str] = Field(
        'en_male_1',
        description='Bark 음성 캐릭터 프리셋 (bark 엔진 전용)',
    )
    emotion: Optional[str] = Field(
        'neutral',
        description='감정 태그: neutral/laugh/sigh (bark 엔진 전용)',
    )
    output_format: Literal['wav', 'mp3', 'base64'] = Field(
        'wav',
        description='출력 형식: wav(스트리밍), mp3(스트리밍), base64(JSON)',
    )
    temperature: float = Field(0.7, ge=0.1, le=1.5)

    model_config = {
        'json_schema_extra': {
            'examples': [{
                'text': 'Hello! Welcome to our AI audio service.',
                'engine': 'speecht5',
```

```

        'output_format': 'wav',
    })
}
}

@router.post(
    '/synthesize',
    summary='텍스트를 음성으로 합성',
    description='텍스트를 입력하면 선택한 TTS 엔진으로 음성 파일을 생성합니다.',
    response_class=StreamingResponse,
    responses={
        200: {'description': 'WAV 또는 MP3 오디오 스트림'},
        422: {'description': '입력 데이터 오류'},
        503: {'description': '모델 미로드'},
    },
)

async def synthesize_speech(request: TTSRequest):
    try:
        # ——— 음성 합성 —————
        if request.engine == 'bark':
            if bark_tts.model is None:
                raise HTTPException(503, 'Bark 모델 미로드')
            audio_array = bark_tts.synthesize(
                text=request.text,
                voice_preset=request.voice_preset or 'en_male_1',
                emotion=request.emotion or 'neutral',
                temperature=request.temperature,
            )
            sample_rate = bark_tts.sample_rate
        else: # speecht5
            if speecht5_tts.model is None:
                raise HTTPException(503, 'SpeechT5 모델 미로드')
            audio_array = speecht5_tts.synthesize(text=request.text)
            sample_rate = speecht5_tts.sample_rate

        # ——— 출력 형식별 응답 —————

```

```
if request.output_format == 'base64':
    buf = io.BytesIO()
    sf.write(buf, audio_array, sample_rate, format='WAV')
    b64 = base64.b64encode(buf.getvalue()).decode()
    return {
        'text': request.text,
        'engine': request.engine,
        'audio_base64': b64,
        'format': 'wav',
        'sample_rate': sample_rate,
        'duration_sec': round(len(audio_array) / sample_rate, 2),
    }

elif request.output_format == 'mp3':
    # pydub으로 MP3 변환
    try:
        from pydub import AudioSegment
        wav_buf = io.BytesIO()
        sf.write(wav_buf, audio_array, sample_rate, format='WAV')
        wav_buf.seek(0)
        audio_seg = AudioSegment.from_wav(wav_buf)
        mp3_buf = io.BytesIO()
        audio_seg.export(mp3_buf, format='mp3', bitrate='128k')
        mp3_buf.seek(0)
        return StreamingResponse(
            mp3_buf,
            media_type='audio/mpeg',
            headers={'Content-Disposition':
                'attachment; filename="speech.mp3"'},
        )
    except ImportError:
        raise HTTPException(
            400, 'MP3 변환에는 pydub 설치 필요'
        )

else: # wav (기본)
    buf = io.BytesIO()
    sf.write(buf, audio_array, sample_rate, format='WAV')
```

```

        buf.seek(0)

    return StreamingResponse(
        buf,
        media_type='audio/wav',
        headers={
            'Content-Disposition': 'attachment; filename="speech.wav"',
            'X-Audio-Duration': str(round(len(audio_array)/sample_rate, 2)),
            'X-Sample-Rate': str(sample_rate),
        },
    )

except HTTPException:
    raise

except Exception as e:
    raise HTTPException(500, f'TTS 처리 오류: {str(e)}')

@router.get('/voices', summary='사용 가능한 음성 프리셋 목록')
async def list_voices():
    return {
        'bark_presets': list(bark_tts.VOICE_PRESETS.keys()),
        'bark_emotions': list(bark_tts.EMOTION_TAGS.keys()),
        'bark_loaded': bark_tts.model is not None,
        'speech5_loaded': speech5_tts.model is not None,
    }

# WAV 스트리밍 테스트
$ curl -X POST http://localhost:8000/tts/synthesize \
-H 'Content-Type: application/json' \
-d '{"text": "Hello, this is a TTS test.", "engine": "speech5"}' \
--output speech.wav

# Base64 응답 테스트
$ curl -X POST http://localhost:8000/tts/synthesize \
-H 'Content-Type: application/json' \
-d '{"text": "Test audio", "engine": "speech5",
    "output_format": "base64"}'

```



```
# → {"audio_base64": "UklGRiQAAABXQVZFZm10IBAAAA...", ...}
```

```
# Base64를 파일로 저장 (Python)
```

```
import base64
```

```
response = requests.post(...)
```

```
data = response.json()
```

```
with open('speech.wav', 'wb') as f:
```

```
    f.write(base64.b64decode(data['audio_base64']))
```

5.3 FastAPI 음성 챗봇 (텍스트 변환 기반)

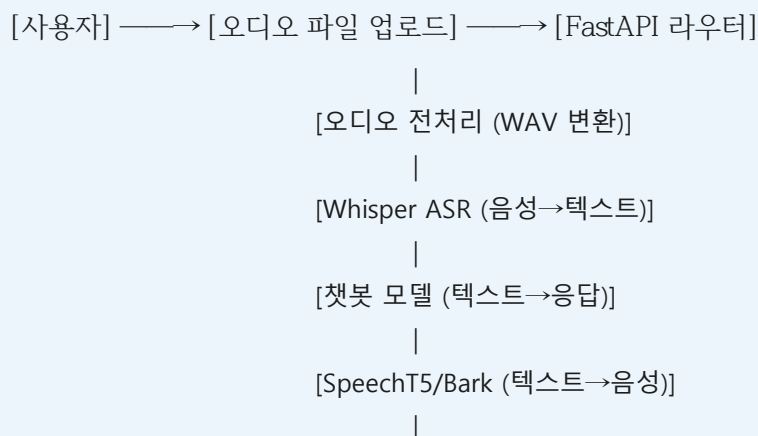
이 절에서는 지금까지 구현한 ASR, 챗봇, TTS 서비스를 하나의 파이프라인으로 연결합니다. 사용자가 음성 파일을 업로드하면 Whisper가 텍스트로 변환하고, LLM이 응답 텍스트를 생성하며, 마지막으로 TTS가 그 응답을 다시 음성으로 변환하는 완전한 음성 챗봇 서비스입니다.

5.3.1 음성 챗봇 전체 흐름: 음성 → 텍스트 → AI 응답 → 음성

음성 챗봇의 전체 데이터 흐름을 단계별로 정리합니다.

처리 단계	수행 내용 및 사용 기술
① 입력 수신	클라이언트가 HTTP POST로 오디오 파일(WAV/MP3) 업로드
② 오디오 전처리	pydub/soundfile로 16kHz 모노 WAV 변환
③ ASR (음성→텍스트)	Whisper로 음성 인식, 텍스트 추출
④ 언어 처리	챗봇 서비스(DialogGPT/KoGPT)로 응답 텍스트 생성
⑤ TTS (텍스트→음성)	SpeechT5 또는 Bark로 응답 텍스트를 음성 합성
⑥ 응답 반환	WAV 오디오 스트리밍 또는 Base64+텍스트 JSON 반환

🔧 음성 챗봇 처리 파이프라인



[사용자] ← [오디오 응답 스트리밍] ← [WAV/MP3/Base64 반환]

5.3.2 챗봇 텍스트 응답 생성 서비스

services/chatbot_service.py (음성 챗봇용 경량 버전)

```
# services/chatbot_service.py (음성 챗봇 통합용)
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM
from typing import Optional

class VoiceChatbotTextService:
    """음성 챗봇용 텍스트 생성 서비스"""

    def __init__(self):
        self.tokenizer = None
        self.model = None
        self.device = 'cuda' if torch.cuda.is_available() else 'cpu'

    def load_model(self, model_id: str = 'microsoft/DialoGPT-small'):
        """
        음성 챗봇에는 DialoGPT-small 사용 권장.
        (음성 처리 비용이 크므로 텍스트 모델은 경량 사용)
        """
        print(f'[ChatBot] Loading model: {model_id}')
        self.tokenizer = AutoTokenizer.from_pretrained(model_id)
        self.model = AutoModelForCausalLM.from_pretrained(
            model_id
        ).to(self.device)
        self.model.eval()
        print('[ChatBot] Loaded.')

    def generate_response(
        self,
        user_text: str,
```

```

        max_length: int = 150,
        temperature: float = 0.8,
    ) -> str:
        if self.model is None:
            # 폴백: 간단한 규칙 기반 응답
            return self._rule_based_response(user_text)

        input_ids = self.tokenizer.encode(
            user_text + self.tokenizer.eos_token,
            return_tensors='pt'
        ).to(self.device)

        with torch.no_grad():
            output_ids = self.model.generate(
                input_ids,
                max_length=max_length,
                pad_token_id=self.tokenizer.eos_token_id,
                do_sample=True,
                temperature=temperature,
                top_p=0.9,
                repetition_penalty=1.3,
            )

        response_ids = output_ids[:, input_ids.shape[-1]:]
        response = self.tokenizer.decode(
            response_ids[0], skip_special_tokens=True
        ).strip()
        return response if response else 'I understand. Please continue.'

    def _rule_based_response(self, text: str) -> str:
        """모델 미로드 시 간단한 규칙 기반 응답"""
        greetings = ['hello', 'hi', 'hey', '안녕', '안녕하세요']
        for g in greetings:
            if g in text.lower():
                return '안녕하세요! 무엇을 도와드릴까요?'
        return '말씀하신 내용을 잘 들었습니다. 더 자세히 말씀해 주시겠어요?'

```

```
voice_chatbot_text = VoiceChatbotTextService()
```

5.3.3 음성 챗봇 통합 라우터 구현

`routers/voice_chat.py`

```
# routers/voice_chat.py
import os
import io
import time
from pathlib import Path
from typing import Optional, Literal

from fastapi import (
    APIRouter, UploadFile, File, Form,
    HTTPException, BackgroundTasks
)
from fastapi.responses import StreamingResponse
from pydantic import BaseModel
import soundfile as sf
import base64

from services.asr_service import asr_service
from services.tts_service import bark_tts, speech5_tts
from services.chatbot_service import voice_chatbot_text
from services.audio_utils import (
    validate_audio_file, save_upload_file, convert_to_wav
)

router = APIRouter(prefix='/voice-chat', tags=['Voice Chatbot'])

class VoiceChatResponse(BaseModel):
    user_speech_text: str    # ASR 변환 텍스트
    bot_response_text: str   # 챗봇 응답 텍스트
    asr_time_sec: float
    chat_time_sec: float
```

```

tts_time_sec: float
total_time_sec: float
audio_base64: Optional[str] = None # base64 모드
audio_format: str = 'wav'

@router.post(
    '/chat',
    summary='음성 챗봇 (음성→텍스트→AI 응답→음성)',
    description='''
사용자의 음성 파일을 업로드하면:
1. Whisper ASR로 텍스트 변환
2. 챗봇 AI로 응답 텍스트 생성
3. TTS로 응답 음성 합성 후 반환
''',
)
async def voice_chat(
    background_tasks: BackgroundTasks,
    file: UploadFile = File(
        ...,
        description='사용자 음성 파일 (wav/mp3, 최대 20MB)'
    ),
    tts_engine: str = Form(
        'speecht5',
        description='TTS 엔진 (bark / speecht5)',
    ),
    response_format: Literal['audio', 'json'] = Form(
        'audio',
        description='응답 형식: audio(WAV 스트리밍), json(텍스트+Base64)',
    ),
    language: Optional[str] = Form(
        None,
        description='ASR 언어 코드 (None=자동감지)',
    ),
):
    total_start = time.time()
    content = await file.read()

```

```
try:
    validate_audio_file(file.filename, len(content))
except ValueError as e:
    raise HTTPException(422, str(e))

# — STEP 1: 파일 저장 및 WAV 변환 —
tmp_path = save_upload_file(content, file.filename)
ext = Path(file.filename).suffix.lower()
if ext != '.wav':
    wav_path = tmp_path.replace(ext, '_conv.wav')
    convert_to_wav(tmp_path, wav_path)
    asr_input = wav_path
else:
    asr_input = tmp_path

try:
    # — STEP 2: ASR —
    asr_start = time.time()
    asr_result = asr_service.transcribe(
        audio_path=asr_input,
        language=language,
    )
    asr_time = round(time.time() - asr_start, 2)
    user_text = asr_result['text']

    if not user_text.strip():
        raise HTTPException(
            422,
            '음성에서 텍스트를 인식하지 못했습니다. 더 명확하게 말씀해 주세요.'
        )

    # — STEP 3: 챗봇 응답 —
    chat_start = time.time()
    bot_text = voice_chatbot_text.generate_response(user_text)
    chat_time = round(time.time() - chat_start, 2)
```

```

# — STEP 4: TTS 합성 —————
tts_start = time.time()
if tts_engine == 'bark' and bark_tts.model:
    audio_array = bark_tts.synthesize(bot_text)
    sample_rate = bark_tts.sample_rate
else:
    audio_array = speech5_tts.synthesize(bot_text)
    sample_rate = speech5_tts.sample_rate
tts_time = round(time.time() - tts_start, 2)

total_time = round(time.time() - total_start, 2)

# — STEP 5: 응답 반환 —————
if response_format == 'json':
    buf = io.BytesIO()
    sf.write(buf, audio_array, sample_rate, format='WAV')
    audio_b64 = base64.b64encode(buf.getvalue()).decode()
    return VoiceChatResponse(
        user_speech_text=user_text,
        bot_response_text=bot_text,
        asr_time_sec=asr_time,
        chat_time_sec=chat_time,
        tts_time_sec=tts_time,
        total_time_sec=total_time,
        audio_base64=audio_b64,
        audio_format='wav',
    )
else: # audio 스트리밍
    buf = io.BytesIO()
    sf.write(buf, audio_array, sample_rate, format='WAV')
    buf.seek(0)
    return StreamingResponse(
        buf,
        media_type='audio/wav',
        headers={
            'X-User-Text': user_text[:100],
            'X-Bot-Text': bot_text[:100],
            'X-Total-Time': str(total_time),

```



```

        'X-ASR-Time': str(asr_time),
        'X-TTS-Time': str(tts_time),
    },
)

except HTTPException:
    raise
except Exception as e:
    raise HTTPException(500, f'음성 챗봇 오류: {str(e)}')
finally:
    background_tasks.add_task(os.unlink, tmp_path)
    if ext != '.wav' and os.path.exists(asr_input):
        background_tasks.add_task(os.unlink, asr_input)

@router.post(
    '/transcribe-only',
    summary='음성→텍스트만 반환 (챗봇 응답 없음)',
)
async def transcribe_only(
    background_tasks: BackgroundTasks,
    file: UploadFile = File(...),
    language: Optional[str] = Form(None),
):
    """디버깅 또는 ASR 단독 테스트용"""
    content = await file.read()
    validate_audio_file(file.filename, len(content))
    tmp = save_upload_file(content, file.filename)
    ext = Path(file.filename).suffix.lower()
    if ext != '.wav':
        wav = tmp.replace(ext, '_c.wav')
        convert_to_wav(tmp, wav)
        process = wav
    else:
        process = tmp
    try:
        result = asr_service.transcribe(process, language=language)

```

```

    return {'text': result['text'], 'language': result['language']}

finally:

    background_tasks.add_task(os.unlink, tmp)

    if ext != '.wav': background_tasks.add_task(os.unlink, process)

# 음성 챗봇 테스트
$ curl -X POST http://localhost:8000/voice-chat/chat ₩
-F 'file=@my_question.wav' ₩
-F 'tts_engine=speecht5' ₩
-F 'response_format=json' ₩
-F 'language=ko'

# JSON 응답 예시:
{
  "user_speech_text": "오늘 날씨가 어떨까요?",
  "bot_response_text": "현재 날씨 정보에 접근하기 어렵지만, 맑은 날 되세요!",
  "asr_time_sec": 3.21,
  "chat_time_sec": 1.05,
  "tts_time_sec": 4.82,
  "total_time_sec": 9.15,
  "audio_base64": "UklGRiQAAABXQVZFZm10IBAAAA...",
  "audio_format": "wav"
}

```

💡 음성 챗봇 성능 최적화 팁

음성 챗봇의 전체 응답 시간은 ASR + 텍스트 생성 + TTS 시간의 합입니다.

각 단계별 최적화 방법:

ASR 최적화:

- Whisper tiny/base 사용 (CPU 환경)
- 오디오를 미리 16kHz로 인코딩하여 전처리 시간 단축

챗봇 텍스트 생성 최적화:

- DialoGPT-small 또는 경량 LLM 사용
- 응답 max_length를 50~ 100 토큰으로 제한

- Hugging Face Inference API 활용 (서버 자원 절약)

TTS 최적화:

- SpeechT5가 Bark보다 10~50배 빠름 (Bark는 GPU 필수)
- 자주 사용되는 응답 문구는 미리 합성하여 캐시

GPU 사용 시 전체 파이프라인 목표:

- Whisper base + DialoGPT-small + SpeechT5 = 약 2~4초 (T4 GPU)

5.4 오디오 파일 업로드 및 처리

이 절에서는 FastAPI의 파일 업로드 처리 기능을 깊이 있게 살펴봅니다. UploadFile의 동작 원리, 대용량 파일 스트리밍 처리, BackgroundTasks를 이용한 비동기 후처리, 그리고 처리된 파일을 다운로드하는 API까지 완성합니다.

5.4.1 FastAPI에서 UploadFile로 .wav, .mp3 파일 처리하기

FastAPI의 파일 업로드는 크게 두 가지 방식으로 이루어집니다. 첫 번째는 UploadFile을 사용하는 multipart/form-data 방식이며, 두 번째는 Request.body()로 바이너리 데이터를 직접 받는 방식입니다. 오디오 파일처럼 큰 파일은 UploadFile 방식을 권장합니다.

UploadFile 내부 동작 원리

FastAPI의 UploadFile은 파이썬의 SpooledTemporaryFile을 래핑한 클래스입니다. 파일 크기가 작으면 메모리에, 크면 임시 파일로 자동 관리합니다.

```
from fastapi import UploadFile, File, Form
from typing import List

# ——— 단일 파일 업로드 ———
@router.post('/upload/single')
async def upload_single(
    file: UploadFile = File(
        ...,
        description='오디오 파일 (WAV, MP3)',
    ),
):
    # 파일 메타데이터 접근
    print(f'파일명: {file.filename}')
    print(f'MIME 타입: {file.content_type}')

    # 전체 내용 읽기 (소용량 파일용)
    content = await file.read()
    file_size = len(content)
```

```
print(f'파일 크기: {file_size / 1024:.1f} KB')

# 파일 포인터 초기화 (재읽기 시 필요)
await file.seek(0)

return {
    'filename': file.filename,
    'content_type': file.content_type,
    'size_bytes': file_size,
}

# ——— 다중 파일 업로드 ———
@router.post('/upload/multiple')
async def upload_multiple(
    files: List[UploadFile] = File(...),
):
    results = []
    for file in files:
        content = await file.read()
        results.append({
            'filename': file.filename,
            'size_bytes': len(content),
            'content_type': file.content_type,
        })
    return {'uploaded': results, 'count': len(results)}

# ——— 파일 + 메타데이터 폼 데이터 ———
@router.post('/upload/with-metadata')
async def upload_with_metadata(
    file: UploadFile = File(...),
    description: str = Form("", description='파일 설명'),
    language: str = Form('ko', description='오디오 언어'),
    auto_transcribe: bool = Form(False, description='자동 전사 여부'),
):
    content = await file.read()
```

```

return {
    'filename': file.filename,
    'description': description,
    'language': language,
    'auto_transcribe': auto_transcribe,
    'size_bytes': len(content),
}

# ——— 대용량 파일 청크 스트리밍 처리 ———
@router.post('/upload/streaming')
async def upload_streaming(
    file: UploadFile = File(...),
):
    """대용량 파일을 청크 단위로 처리 (메모리 절약)"""
    import uuid
    save_path = f'temp/{uuid.uuid4()}{Path(file.filename).suffix}'
    total_size = 0
    chunk_size = 1024 * 1024 # 1MB 청크

    with open(save_path, 'wb') as f:
        while chunk := await file.read(chunk_size):
            f.write(chunk)
            total_size += len(chunk)

    return {
        'saved_path': save_path,
        'total_size_mb': round(total_size / 1024 / 1024, 2)
    }

```

5.4.2 업로드된 파일을 서버 임시 저장 → 모델에 전달 → 결과 반환

파일 처리 파이프라인에서 임시 파일 관리는 매우 중요합니다. 처리가 완료된 임시 파일은 반드시 삭제하여 디스크 공간을 낭비하지 않아야 하며, BackgroundTasks를 활용하면 응답을 먼저 반환한 후 정리 작업을 수행할 수 있습니다.

파일 관리 시점	처리 방법
업로드 즉시	save_upload_file()로 temp/ 디렉터리에 UUID 기반 파일명으로 저장
포맷 변환	convert_to_wav()로 변환 후 원본 임시파일 즉시 삭제
모델 처리 중	처리 중인 파일은 유지 (try/finally 블록으로 보장)
응답 반환 직후	BackgroundTasks.add_task(os.unlink, path)로 비동기 삭제
주기적 정리	cleanup_temp_files()를 스케줄러 또는 startup 이벤트에 등록

완전한 오디오 처리 라우터 구현

```
# routers/audio_upload.py

import os
import uuid
import shutil
import time
from pathlib import Path
from typing import Optional, List

from fastapi import (
    APIRouter, UploadFile, File, Form,
    HTTPException, BackgroundTasks, status
)
from fastapi.responses import FileResponse, StreamingResponse
from pydantic import BaseModel

from services.audio_utils import (
    validate_audio_file, convert_to_wav,
    get_audio_info, save_upload_file,
    SUPPORTED_AUDIO_FORMATS, OUTPUT_DIR, TEMP_DIR
)

router = APIRouter(prefix='/audio', tags=['Audio File Management'])

# 처리된 파일 목록 (메모리 내 간단 레지스트리)
processed_files: dict = {}
```

```

class AudioFileInfo(BaseModel):
    file_id: str
    filename: str
    duration_sec: Optional[float]
    sample_rate: Optional[int]
    channels: Optional[int]
    size_bytes: int
    format: str
    status: str # 'uploaded' | 'processing' | 'done' | 'error'

@router.post(
    '/upload',
    response_model=AudioFileInfo,
    summary='오디오 파일 업로드',
    description='오디오 파일을 서버에 업로드하고 파일 정보를 반환합니다.',
)
async def upload_audio(
    file: UploadFile = File(
        ...,
        description='오디오 파일. 지원 형식: {SUPPORTED_AUDIO_FORMATS}',
    ),
    convert_to_wav_flag: bool = Form(
        True,
        description='True면 16kHz 모노 WAV로 자동 변환',
    ),
):
    content = await file.read()
    size = len(content)

    try:
        validate_audio_file(file.filename, size)
    except ValueError as e:
        raise HTTPException(422, str(e))

    # 저장

```



```
file_id = str(uuid.uuid4())
ext = Path(file.filename).suffix.lower()
save_path = str(TEMP_DIR / f'{file_id}{ext}')
with open(save_path, 'wb') as f:
    f.write(content)

# WAV 변환
if convert_to_wav_flag and ext != '.wav':
    wav_path = str(TEMP_DIR / f'{file_id}.wav')
    convert_to_wav(save_path, wav_path)
    os.unlink(save_path)
    final_path = wav_path
    final_ext = '.wav'
else:
    final_path = save_path
    final_ext = ext

audio_info = get_audio_info(final_path)
processed_files[file_id] = {
    'path': final_path,
    'original_name': file.filename,
    'status': 'uploaded',
    'uploaded_at': time.time(),
}

return AudioFileInfo(
    file_id=file_id,
    filename=file.filename,
    duration_sec=audio_info.get('duration_sec'),
    sample_rate=audio_info.get('sample_rate'),
    channels=audio_info.get('channels'),
    size_bytes=size,
    format=final_ext.lstrip('.'),
    status='uploaded',
)
```

```
@router.get(
    '/download/{file_id}',
    summary='처리된 오디오 파일 다운로드',
    response_class=FileResponse,
)

async def download_audio(file_id: str):
    if file_id not in processed_files:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f'파일 ID {file_id}를 찾을 수 없습니다.',
        )
    file_info = processed_files[file_id]
    file_path = file_info['path']

    if not os.path.exists(file_path):
        del processed_files[file_id]
        raise HTTPException(404, '파일이 서버에 존재하지 않습니다.')

    return FileResponse(
        path=file_path,
        media_type='audio/wav',
        filename=f'processed_{file_info["original_name"]}',
    )

@router.delete('/delete/{file_id}', summary='업로드 파일 삭제')
async def delete_audio(file_id: str):
    if file_id not in processed_files:
        raise HTTPException(404, '파일을 찾을 수 없습니다.')
    info = processed_files.pop(file_id)
    path = info['path']
    if os.path.exists(path):
        os.unlink(path)
    return {'message': f'파일 {file_id} 삭제 완료'}

@router.get('/list', summary='업로드된 파일 목록')
```

```

async def list_audio_files():
    result = []
    for fid, info in processed_files.items():
        result.append({
            'file_id': fid,
            'original_name': info['original_name'],
            'status': info['status'],
            'path_exists': os.path.exists(info['path']),
        })
    return {'files': result, 'total': len(result)}

```

5.4.3 업로드 후 처리 API + 다운로드 API 구현 예제

업로드된 파일을 특정 작업(포맷 변환, 음성 향상, 구간 추출 등)으로 처리한 뒤, 처리된 파일을 다운로드할 수 있는 완전한 파이프라인을 구현합니다.

```

# routers/audio_upload.py (처리 엔드포인트 추가)

@router.post(
    '/process/{file_id}',
    summary='업로드 파일 처리 (포맷 변환, 구간 추출 등)',
)
async def process_audio(
    file_id: str,
    background_tasks: BackgroundTasks,
    operation: str = Form(
        'convert',
        description='처리 유형: convert(변환)/trim(구간추출)/normalize(정규화)',
    ),
    start_sec: float = Form(0.0, description='구간 추출 시작 시간(초)'),
    end_sec: float = Form(0.0, description='구간 추출 종료 시간(초). 0=끝까지'),
    target_sample_rate: int = Form(16000, description='목표 샘플링 레이트'),
):
    if file_id not in processed_files:
        raise HTTPException(404, '파일을 찾을 수 없습니다.')

    info = processed_files[file_id]

```

```
input_path = info['path']

if not os.path.exists(input_path):
    raise HTTPException(404, '파일이 서버에 없습니다.')

try:
    import soundfile as sf
    import numpy as np

    data, sr = sf.read(input_path)
    output_id = str(uuid.uuid4())
    output_path = str(OUTPUT_DIR / f'{output_id}.wav')

    if operation == 'trim':
        # 구간 추출
        s_sample = int(start_sec * sr)
        e_sample = int(end_sec * sr) if end_sec > 0 else len(data)
        data = data[s_sample:e_sample]

    elif operation == 'normalize':
        # 볼륨 정규화 (RMS 기반)
        rms = np.sqrt(np.mean(data**2))
        if rms > 0:
            data = data / rms * 0.1

    elif operation == 'convert':
        # 샘플링 레이트 변환
        if sr != target_sample_rate:
            import scipy.signal as signal
            n = int(len(data) * target_sample_rate / sr)
            data = signal.resample(data, n)
            sr = target_sample_rate

    sf.write(output_path, data, sr)

    # 처리된 파일 등록
```

```

processed_files[output_id] = {
    'path': output_path,
    'original_name': f'processed_{info["original_name"]}',
    'status': 'done',
    'uploaded_at': time.time(),
}

return {
    'source_file_id': file_id,
    'output_file_id': output_id,
    'operation': operation,
    'download_url': f'/audio/download/{output_id}',
    'audio_info': get_audio_info(output_path),
}

except Exception as e:
    raise HTTPException(500, f'파일 처리 오류: {str(e)}')

```

5.4.4 File, Form, BackgroundTasks 사용법 포함

FastAPI에서 파일과 폼 데이터를 함께 처리할 때 주의할 점과 BackgroundTasks의 다양한 활용 패턴을 정리합니다.

FastAPI 파라미터 타입	역할 및 사용 상황
File(...)	필수 파일 업로드 파라미터. UploadFile과 함께 사용.
File(None)	선택적 파일 업로드. Optional[UploadFile]과 함께 사용.
Form(...)	필수 폼 데이터. multipart/form-data 요청에서 텍스트 파라미터.
Form(default)	선택적 폼 데이터. 기본값과 함께 사용.
BackgroundTasks	응답 반환 후 실행될 작업 등록. 임시 파일 삭제, 알림 전송 등.

⚠ 파일 + JSON Body 동시 사용 불가

FastAPI에서 UploadFile과 Pydantic BaseModel(JSON Body)은 같은 엔드포인트에 동시에 사용할 수 없습니다. (multipart/form-data vs application/json 충돌)

해결책 1: 모든 파라미터를 Form()으로 정의

```
async def endpoint(
    file: UploadFile = File(...),
    name: str = Form(...),
    language: str = Form('ko'),
):
```

해결책 2: JSON을 문자열 폼으로 전달 후 파싱

```
async def endpoint(
    file: UploadFile = File(...),
    metadata: str = Form('{}'), # JSON 문자열
):
    import json
    data = json.loads(metadata)
```

해결책 3: 파일 업로드와 설정을 별도 엔드포인트로 분리

BackgroundTasks 고급 활용 패턴

```
from fastapi import BackgroundTasks
import asyncio
import logging

logger = logging.getLogger(__name__)

# ——— 패턴 1: 파일 삭제 (가장 일반적) ———
async def cleanup(path: str):
    try:
        os.unlink(path)
        logger.info(f'Cleared: {path}')
    except FileNotFoundError:
        pass

@router.post('/upload')
async def upload(bt: BackgroundTasks, file: UploadFile = File(...)):
```

```

content = await file.read()
tmp = save_upload_file(content, file.filename)
result = process(tmp)
bt.add_task(cleanup, tmp) # 응답 후 삭제
return result

# —— 패턴 2: 비동기 후처리 (결과 저장, 알림 등) ——
async def post_process(file_id: str, text: str):
    """전사 결과를 DB에 저장하거나 알림 발송"""
    await asyncio.sleep(0) # 이벤트 루프 양보
    logger.info(f'Post-processing {file_id}: {text[:50]}')
    # await db.save(file_id, text)
    # await notify_webhook(file_id, text)

@router.post('/transcribe-async')
async def transcribe_async(
    bt: BackgroundTasks,
    file: UploadFile = File(...),
):
    content = await file.read()
    tmp = save_upload_file(content, file.filename)
    result = asr_service.transcribe(tmp)

    # 동기 함수도 BackgroundTasks로 실행 가능
    bt.add_task(os.unlink, tmp)
    # 비동기 함수도 실행 가능
    bt.add_task(post_process, str(uuid.uuid4()), result['text'])

    return {'text': result['text']}

# —— 패턴 3: 정기 임시 파일 정리 ——
from apscheduler.schedulers.asyncio import AsyncIOScheduler

scheduler = AsyncIOScheduler()

@app.on_event('startup')

```

```
async def start_scheduler():
    scheduler.add_job(
        cleanup_temp_files,
        'interval',
        hours=1,
        kwargs={'max_age_sec': 3600}
    )
    scheduler.start()
```

TIP: UploadFile 완전 정리

FastAPI UploadFile 사용 시 꼭 알아야 할 핵심 사항:

1. 항상 await file.read()로 비동기 읽기
 - file.read() (동기) 사용 시 블로킹 발생
2. 대용량 파일은 청크 읽기 사용
 - while chunk := await file.read(1024*1024): ...
3. multipart/form-data 헤더 필수
 - Swagger UI는 자동 처리
 - curl: -F 'file=@path.wav' (Content-Type 자동 설정)
 - Python requests: files={'file': open('audio.wav', 'rb')}
4. 파일명 인코딩 주의
 - 한글 파일명은 urllib.parse.unquote(file.filename) 처리 권장
5. content_type 신뢰 금지
 - 클라이언트가 임의 설정 가능. 파일 확장자로 검증 권장.

5.5 통합 main.py 및 전체 서비스 실행

지금까지 구현한 모든 서비스를 하나의 FastAPI 앱으로 통합합니다. lifespan 이벤트 핸들러로 모델을 초기화하고, 각 라우터를 등록하는 완성된 main.py를 작성합니다.

```
# main.py
import os
from contextlib import asynccontextmanager
from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse
import logging

# 로거 설정
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s | %(name)-20s | %(levelname)-8s | %(message)s',
    datefmt='%Y-%m-%d %H:%M:%S',
)
logger = logging.getLogger(__name__)

# 서비스 임포트
from services.asr_service import asr_service
from services.tts_service import bark_tts, speech5_tts
from services.chatbot_service import voice_chatbot_text
from services.audio_utils import cleanup_temp_files

# 라우터 임포트
from routers import asr, tts, voice_chat, audio_upload

@asynccontextmanager
async def lifespan(app: FastAPI):
    """서버 시작/종료 시 실행되는 이벤트 핸들러"""
    logger.info('=' * 60)
```

```
logger.info('Audio AI Services Starting...')
logger.info('=' * 60)

# 환경변수로 로드할 모델 제어
if os.getenv('LOAD_ASR', 'true').lower() == 'true':
    asr_service.load_model()

if os.getenv('LOAD_SPEECHT5', 'true').lower() == 'true':
    speecht5_tts.load_model()

if os.getenv('LOAD_BARK', 'false').lower() == 'true':
    bark_tts.load_model() # 메모리 큼, 선택적 로드

if os.getenv('LOAD_CHATBOT', 'true').lower() == 'true':
    voice_chatbot_text.load_model()

# 임시 파일 초기 정리
cleanup_temp_files(max_age_sec=0) # 이전 실행 파일 전체 삭제

logger.info('All models loaded. Server ready!')
logger.info('Docs: http://localhost:8000/docs')
yield

# 종료 시 정리
logger.info('Shutting down...')
cleanup_temp_files(max_age_sec=0)

app = FastAPI(
    title='Audio AI Web Services',
    description=""
    ## 오디오 AI 웹 서비스 API

    Hugging Face 오디오 AI 모델 기반 FastAPI 서비스.
```

```

### 제공 서비스
- **ASR** (음성 인식): Whisper 기반 다국어 음성→텍스트 변환
- **TTS** (음성 합성): Bark / SpeechT5 기반 텍스트→음성 변환
- **Voice Chat** (음성 챗봇): ASR + LLM + TTS 파이프라인
- **File Manager** (파일 관리): 오디오 파일 업로드/처리/다운로드

'''
version='1.0.0',
lifespan=lifespan,
)

# CORS 미들웨어 (프론트엔드 연동 시 필요)
app.add_middleware(
    CORSMiddleware,
    allow_origins=['*'],
    allow_credentials=True,
    allow_methods=['*'],
    allow_headers=['*'],
)

# 전역 예외 핸들러
@app.exception_handler(Exception)
async def global_exception_handler(request: Request, exc: Exception):
    logger.error(f'Unhandled error: {exc}', exc_info=True)
    return JSONResponse(
        status_code=500,
        content={'detail': '서버 내부 오류가 발생했습니다.'},
    )

# 라우터 등록
app.include_router(asr.router)
app.include_router(tts.router)
app.include_router(voice_chat.router)
app.include_router(audio_upload.router)

@app.get('/', tags=['Health'])
async def root():
    return {

```

```

'service': 'Audio AI Web Services',
'version': '1.0.0',
'models': {
    'asr': asr_service.pipe is not None,
    'speecht5': speecht5_tts.model is not None,
    'bark': bark_tts.model is not None,
    'chatbot': voice_chatbot_text.model is not None,
},
'docs': '/docs',
'redoc': '/redoc',
}

if __name__ == '__main__':
    import uvicorn
    uvicorn.run(
        'main:app',
        host='0.0.0.0',
        port=8000,
        reload=True,
        log_level='info',
    )

```

.env 환경변수 설정

```

# .env

# Hugging Face 설정
HF_TOKEN=hf_XXXXXXXXXXXXXXXXXXXXXXXXX
HF_HOME=./hf_cache
TRANSFORMERS_OFFLINE=0

# 모델 로드 제어 (true/false)
LOAD_ASR=true
LOAD_SPEECHT5=true
LOAD_BARK=false    # GPU 환경에서만 true 권장
LOAD_CHATBOT=true

# Whisper 모델 크기

```

```
WHISPER_MODEL_SIZE=base # tiny/base/small/medium/large
```

```
# 서버 설정
```

```
PORT=8000
```

```
ENV=development
```

```
# 파일 제한
```

```
MAX_UPLOAD_SIZE_MB=50
```

```
TEMP_FILE_TTL_SEC=3600
```

```
# 서버 실행
```

```
$ uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

```
# 전체 API 엔드포인트 목록:
```

```
# — ASR (음성 인식) —
```

```
# POST /asr/transcribe      음성 파일 → 텍스트
```

```
# POST /asr/generate-srt    음성 파일 → SRT 자막
```

```
# GET /asr/health          모델 상태 확인
```

```
#
```

```
# — TTS (음성 합성) —
```

```
# POST /tts/synthesize      텍스트 → 음성
```

```
# GET /tts/voices           음성 프리셋 목록
```

```
#
```

```
# — Voice Chat (음성 챗봇) —
```

```
# POST /voice-chat/chat     음성→텍스트→AI→음성
```

```
# POST /voice-chat/transcribe-only ASR 단독 테스트
```

```
#
```

```
# — Audio File (파일 관리) —
```

```
# POST /audio/upload        파일 업로드
```

```
# POST /audio/process/{id}  파일 처리
```

```
# GET /audio/download/{id}  파일 다운로드
```

```
# DELETE /audio/delete/{id} 파일 삭제
```

```
# GET /audio/list           업로드 파일 목록
```

```
#
```

```
# — 문서 —
```

```
# GET /docs                 Swagger UI
```

```
# GET /redoc                 ReDoc
```

🔗 5장 학습 정리

Whisper 모델의 아키텍처와 5가지 크기 버전을 이해하고 ASR API를 완성했습니다.

SRT 자막 자동 생성 기능을 타임스탬프 청크 기반으로 구현했습니다.

Bark와 SpeechT5 두 가지 TTS 엔진을 비교하고, WAV/MP3/Base64 출력을 지원했습니다.

ASR + 챗봇 + TTS를 연결한 완전한 음성 챗봇 파이프라인을 구현했습니다.

UploadFile의 동작 원리와 청크 처리, 폼 데이터 처리 방법을 심층 이해했습니다.

BackgroundTasks로 응답 후 임시 파일을 비동기 정리하는 패턴을 마스터했습니다.

FileResponse와 StreamingResponse로 파일 다운로드 API를 구현했습니다.

CORS 미들웨어, 전역 예외 핸들러, lifespan 이벤트를 활용한 완성도 높은 앱을 완성했습니다.

연습 문제

다음 문제들을 통해 5장의 핵심 개념을 확인합니다.

7. Whisper 모델이 오디오를 처리하는 내부 과정을 6단계로 설명하고, 'realtime_factor'가 1.0보다 낮을 때의 의미를 서술하세요.
8. Bark와 SpeechT5를 비교하여 어떤 환경에서 각각을 사용하는 것이 적합한지 3가지 기준으로 설명하세요.
9. 음성 챗봇 파이프라인에서 각 단계(ASR, 텍스트 생성, TTS)별로 응답 시간을 줄이기 위한 최적화 방법을 각 2가지 이상 제시하세요.
10. FastAPI에서 UploadFile을 사용할 때 아래 코드의 문제점을 찾고 수정하세요.

```
@router.post('/upload')
def upload_file(file: UploadFile = File(...)): # 문제 1
    content = file.read() # 문제 2
    save_path = f'temp/{file.filename}' # 문제 3
    with open(save_path, 'wb') as f:
        f.write(content)
    return {'size': len(content)}
```

11. BackgroundTasks를 사용하지 않고 응답 직후 임시 파일을 삭제하면 어떤 문제가 발생할 수 있나요? 구체적인 시나리오를 들어 설명하세요.
12. [코딩 실습] POST /asr/transcribe-batch 엔드포인트를 구현하세요. 여러 오디오 파일을 동시에 업로드하면 각각 Whisper로 전사하고, 결과를 파일명과 전사 텍스트 쌍의 목록으로 반환합니다. (힌트: List[UploadFile] 활용)

5장 요약

이 장에서는 오디오 AI 기술을 FastAPI와 결합하여 4가지 실용적인 서비스를 구현했습니다.

- Whisper ASR API: OpenAI Whisper 모델을 Hugging Face Pipeline으로 래핑하여 WAV, MP3 등 다양한 포맷의 음성 파일을 텍스트로 변환하는 API를 구현했습니다. 타임스탬프 기반 SRT 자막 자동 생성 기능도 추가했습니다.
- TTS API: Bark(고품질, 감정 표현 가능)와 SpeechT5(빠른 처리)를 모두 구현하고 WAV 스트리밍, MP3 변환, Base64 JSON 세 가지 응답 형식을 지원했습니다.
- 음성 챗봇: Whisper ASR → DialoGPT 챗봇 → TTS 세 단계 파이프라인을 하나의 엔드포인트로 구성했습니다. 응답 헤더에 처리 시간 메타데이터를 포함하여 성능 모니터링을 지원합니다.
- 오디오 파일 관리: UploadFile의 심층 활용, 청크 스트리밍 읽기, File/Form 혼합 처리, BackgroundTasks 기반 비동기 파일 정리, FileResponse 다운로드 API를 구현했습니다.

□ 다음 장 예고: 제6장 이미지 생성 AI 웹 프로젝트

6장에서는 Stable Diffusion, DALL·E API, ControlNet 등을 FastAPI와 연동합니다.

주요 내용:

- Stable Diffusion WebUI API 연동
- text-to-image: 텍스트 프롬프트로 이미지 생성 API
- image-to-image: 이미지+프롬프트로 이미지 변환 API

- ControlNet: 포즈/엣지 기반 이미지 생성 API
- 이미지 스트리밍 응답과 Base64 반환
- 생성 이미지 S3/GCS 저장 및 URL 반환
- 배치 이미지 생성과 큐 기반 비동기 처리